International Journal for Multidisciplinary Research (IJFMR)
E-ISSN: 2582-2160 ● Website: www.ijfmr.com ● Email: editor@ijfmr.com

Agentic AI Meets Deterministic Tooling: A Cost, Speed, and Reliability Analysis of Hybrid Automation for Software Engineering

Umesh Kumar

Hansi, Haryana, India

Abstract

Agentic AI systems, that is, large-language-model systems that dynamically direct their own process and tool use, can complete real software-engineering tasks end to end, but every step an agent takes costs tokens, takes seconds, and succeeds only probabilistically. Deterministic tools such as scripts, state machines, guards, compilers, and test suites cost nothing per run, execute in milliseconds, and behave identically every time, but cannot exercise judgment. In this paper, the architecture that combines them is analysed: an agentic core doing only the work that genuinely requires judgment, wrapped in a deterministic shell that handles dispatch, state tracking, policy enforcement, verification, and lifecycle. The research line from tool-augmented LLMs (ReAct, Toolformer, PAL) and agent-computer-interface design (SWE-agent) to current industry guidance that explicitly distinguishes predefined workflows from dynamic agents is surveyed, and the division-of-labour rule the literature implies is formalised. A parameterised cost model, anchored in current model pricing and in the operational envelope of a production enterprise pipeline, shows the hybrid pattern cutting model-token cost per ticket by roughly 48% and orchestration overhead time by more than two-thirds, while a compounding-reliability analysis shows why replacing probabilistic steps with deterministic ones (and bounding the rest with verifier loops) is the difference between a demo and a dependable system. A reference architecture, an execution-loop pattern, a determinism-placement decision matrix, and a benefits and limitations assessment are contributed for practitioners.

Keywords: Agentic AI, Deterministic Tooling, LLM Agents, Workflows, Orchestration, Cost Model, Reliability, Software Engineering Automation, Tool Use, Verification

1. Introduction

Two automation traditions are converging on the software-engineering task. The older one is deterministic: scripts, build systems, git hooks, CI pipelines, which are code paths defined in advance that run in milliseconds, cost nothing per execution, and do exactly the same thing every time. The newer one is agentic: LLM systems that “dynamically direct their own processes and tool usage” [1], able to read a ticket, plan a change, edit code, and react to failures, which are capabilities no fixed script has. Industry experience is now clear that these are complements, not competitors: Anthropic's engineering guidance distinguishes workflows (LLMs and tools orchestrated through predefined code paths) from agents and recommends “finding the simplest solution possible,” noting that agentic systems

"trade latency and cost for better task performance" [1]; OpenAI's guide to building agents likewise reserves agents for workflows where judgment is essential and wraps them in guardrails [2].

What the guidance lacks is quantification and architecture: how much does it actually cost to let an agent do everything, versus letting it do only what needs judgment? Where exactly should the deterministic/agentic boundary sit, and what does the resulting system look like in production? These questions matter because the costs are real (frontier coding models price output tokens at USD 5 to 50 per million [10]) and because practitioner trust is fragile: 46% of developers actively distrust AI output accuracy [11], and a randomised trial found experienced developers slower with AI tools while believing the opposite [12].

In this paper the hybrid pattern is analysed directly. The contributions are: (i) a consolidated review of the research and industry line that leads to hybrid agent-tool systems (Section 2); (ii) a reference architecture, an agentic core inside a deterministic shell, with an execution-loop pattern and a determinism-placement decision matrix (Sections 3 and 4); (iii) a parameterised cost model anchored in current token pricing [10] and in the operational envelope of a production enterprise agent pipeline operated by the author, showing about 48% token-cost reduction and more than two-thirds orchestration-time reduction per ticket (Section 5); (iv) a reliability analysis explaining, via error compounding and verifier loops, why hybrid systems are dependable where all-agentic chains are not (Section 6); and (v) a benefits and limitations assessment (Section 7).

2. Background and Related Work

2.1 From Tool Use to Agents

The research line begins with making LLMs use deterministic machinery. ReAct interleaves reasoning traces with actions against external tools, improving both task success and interpretability [3]. Toolformer showed models can teach themselves when to call APIs (calculators, search, translation), offloading what the model itself does unreliably [4]. PAL made the division of labour explicit: the LLM writes a program, and a deterministic Python interpreter executes it, outperforming chain-of-thought reasoning on arithmetic and symbolic benchmarks precisely because execution is not left to the stochastic component [5]. SWE-agent generalised the lesson to software engineering: the design of the deterministic agent-computer interface (structured file viewers, guarded edit commands with integrated lint feedback) materially changes agent success on real GitHub issues [6], as benchmarked by SWE-bench [7].

2.2 Agentic Products and Their Deterministic Skeletons

Production agent systems already embody the hybrid quietly. Codex runs each task in a sandbox whose lifecycle, network policy, and log-citation machinery are deterministic scaffolding around the model [8]. GitHub's Copilot cloud agent executes inside an ephemeral CI-managed environment and channels all output through branch and pull-request machinery [9]. Devin pairs the agent with a managed shell, editor and browser workspace and structured progress reporting [13]. Orchestration frameworks such as LangGraph exist precisely to let developers pin the control flow as a graph (deterministic edges) around model-driven nodes [17]. What has been missing is an explicit analysis of why this shape wins and how much it saves; that is the gap this paper addresses.

2.3 The Pressure of Evidence

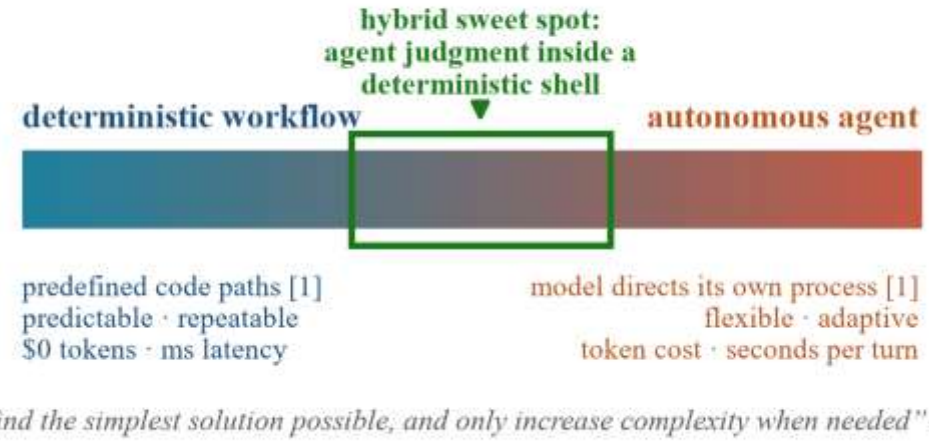
Two 2025 data points sharpen the motivation. Developers' trust in agent output is low (46% distrust versus 33% trust [11]), which makes deterministic verification (compilers, tests, CI) the only scalable

substitute for line-by-line human vigilance. The METR randomised trial [12] showed that perceived AI productivity can exceed reality by about 40 percentage points, which argues for architectures whose costs and outcomes are measurable by construction, exactly what deterministic shells provide (every dispatch, verdict, and state transition is a logged, reproducible event).

3. Definitions and the Hybrid Pattern

A deterministic tool is a component whose behaviour is a pure function of its explicit inputs: scripts, state machines, git hooks, compilers, linters, test runners, schedulers. Its properties are zero marginal token cost, millisecond-to-second latency, reproducibility, and testability. An agentic step is an LLM turn that interprets context and chooses its own next action [1, 2]. Its properties are judgment, adaptation, and natural-language competence, purchased at per-token prices [10], seconds of latency per turn, and probabilistic success. Figure 1 places the two on a spectrum and marks the design point this paper analyses: agent judgment inside a deterministic shell.

Figure 1: The Workflow-Agent Spectrum and the Hybrid Sweet Spot



The division-of-labour rule that Sections 5 and 6 will justify economically is: everything repeatable becomes a deterministic tool; everything requiring judgment remains agentic; everything verifiable gets a deterministic verifier around its agentic producer. The rule echoes PAL's finding at task level [5] and SWE-agent's at interface level [6], and it operationalises the "simplest solution possible" advice of [1].

4. Reference Architecture: How It Works

4.1 An Agentic Core in a Deterministic Shell

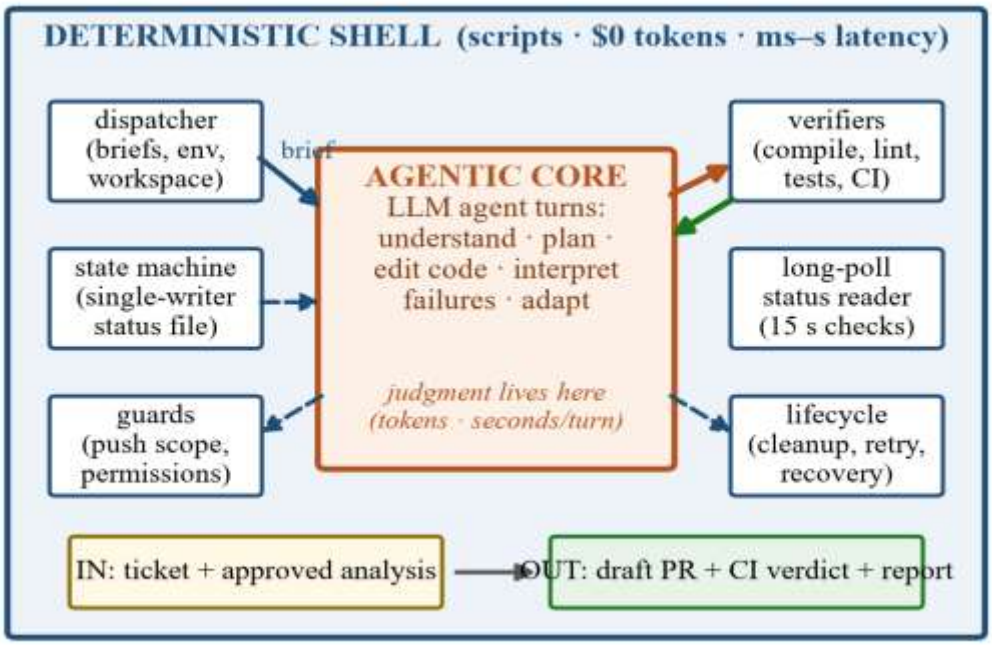
Figure 2 shows the reference architecture, instantiated throughout this section with the production enterprise pipeline the author operates (an autonomous ticket-to-pull-request system for brownfield Java/Angular repositories). The shell owns six responsibilities (Table 1): dispatch, state, guards, verification, observation, and lifecycle. The core receives a precise mission brief and spends tokens only

on judgment: understanding the ticket's analysis document, deriving an implementation story following a structured method [16], editing code, and interpreting verifier feedback.

Table 1: Shell Components and the Agentic Turns They Replace

Shell Component	Deterministic Implementation	Agentic Turns Displaced
dispatcher	script: sync docs repo, write briefs + initial state, spawn detached worker (about 1 min)	about 20 turns of setup tool-calls
state machine	single-writer state file; PENDING to RUNNING to DONE/WARNING per session	self-reported (unreliable) progress
guards	git pre-push hook: feature branches only, protected branches blocked, reference clones read-only	per-turn policy reminders in context
verifiers	compiler, linter, unit/e2e suites, CI on a draft pull request	agent self-review turns
status reader	long-poll: blocks up to 8 min, re-checks state + worker PID every 15 s	poll-every-2-min conversation turns
lifecycle	idempotent cleanup, per-ticket namespacing, crash detection via PID liveness	ad-hoc recovery reasoning

Figure 2: Reference Architecture with a Deterministic Shell Around the Agentic Core

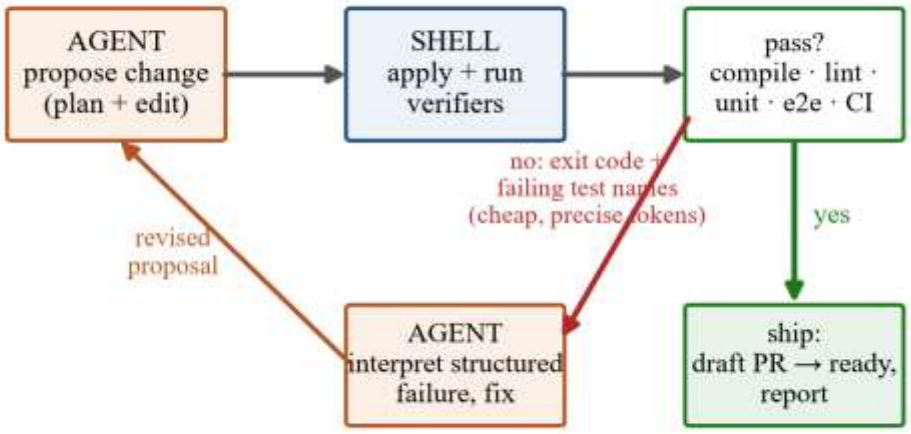


4.2 The Hybrid Execution Loop

Figure 3 shows the inner loop that produces the artefacts. The agent proposes (plan + edit); the shell applies and verifies deterministically; the pass/fail verdict is computed by machinery whose false-

positive rate is effectively zero for what it checks (compilation, lint rules, test assertions, CI). On failure the agent receives a structured, minimal error (exit code, failing test names, diff context) rather than re-reading the world, which keeps feedback tokens small and precise; this is the agent-computer-interface insight of [6] applied to the whole pipeline. The loop terminates on a deterministic condition (CI green), not on the model's own opinion of doneness, and ships via a draft-to-ready pull-request transition that a human ultimately merges.

Figure 3: The Hybrid Execution Loop



tokens are spent only in the orange boxes; the oracle that decides "done" is deterministic

4.3 Where to Draw the Line

Figure 4 gives the placement rule as a decision matrix over two questions: can the step be specified precisely in advance, and does it need judgment? Fully specifiable, judgment-free steps (dispatch, polling, cleanup, guard enforcement, secret handling) are shell work; making them agentic buys nothing but cost and variance. Judgment-heavy, hard-to-specify steps (understanding a ticket, designing a fix, interpreting a novel failure) are core work; scripting them produces brittle automation. The most productive quadrant is judgment with a specifiable success criterion (write code until the tests pass), which is exactly the agent-plus-verifier loop of Figure 3. Steps that are neither specifiable nor judgment-bearing usually indicate an unclear requirement and belong back with a human.

Figure 4: Determinism-Placement Decision Matrix



5. Cost Analysis

5.1 Model

Let an agentic turn consume T_{in} input and T_{out} output tokens at prices p_{in}, p_{out} per million. Task cost for the agentic share is given by Equation (1), while every step moved into the shell contributes zero to Equation (1) and milliseconds to wall time.

$$C = \Sigma_{turns} (T_{in} \times p_{in} + T_{out} \times p_{out}) \quad (1)$$

The model is parameterised with a mid-tier coding model at p_{in} = USD 3 and p_{out} = USD 15 per million tokens (current list prices span USD 1 to 10 for input and USD 5 to 50 for output across tiers [10]) and an average turn of 12k input + 1.2k output tokens, i.e., about USD 0.054 per turn.

5.2 A Representative Ticket

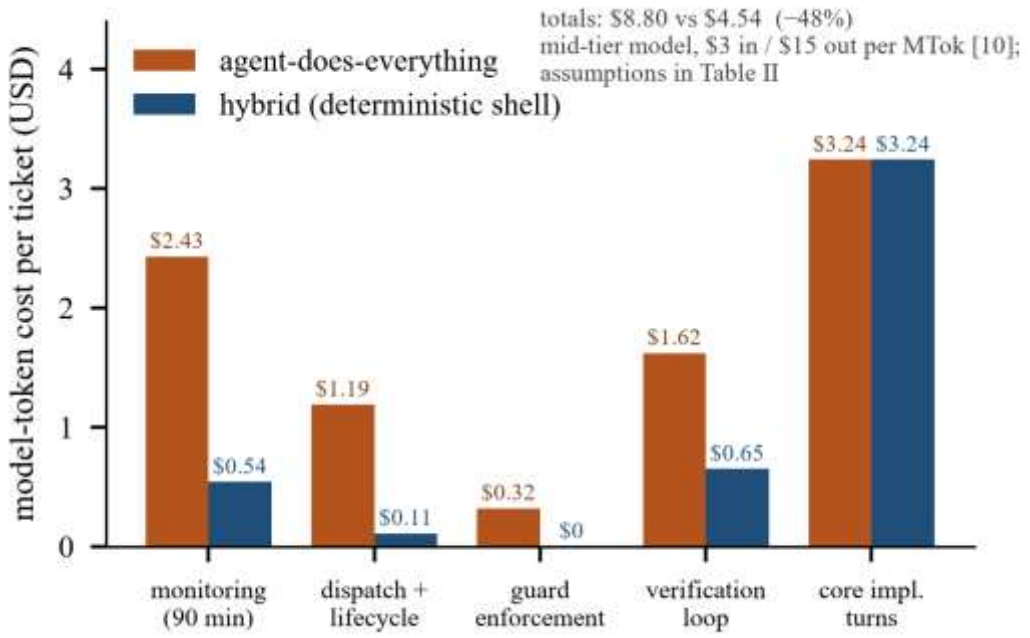
Table 2 decomposes a representative 90-minute ticket in the production pipeline into five components, comparing an agent-does-everything baseline with the hybrid shell. The numbers for the hybrid column are the pipeline's operational envelope: dispatch is one script (about 1 min) invoked in a single turn; monitoring uses the long-poll reader, about 7 status calls per hour instead of a 2-minute conversational poll; guard enforcement is a git hook (zero turns); verification is compiler, test and CI runs whose only agentic cost is interpreting failures. Figure 5 visualises the result: about USD 8.80 versus USD 4.54 per ticket, a 48% reduction, with the entire saving coming from displaced orchestration, not from touching the core implementation turns.

Table 2: Cost Decomposition of a 90-Minute Ticket

Component	Agentic-Only (Turns → USD)	Hybrid (Turns → USD)
monitoring (90 min)	45 → $2.43	10 → $0.54
dispatch + lifecycle	22 → $1.19	2 → $0.11

guard enforcement	6 → $0.32	0 → $0 (git hook)
verification loop	30 → $1.62	12 → $0.65
core implementation	60 → $3.24	60 → $3.24
total	163 → $8.80	84 → $4.54 (−48%)

Figure 5: Token Cost per Ticket by Component



5.3 Sensitivity and Second-Order Effects

The saving is robust to model choice: it is proportional to price-per-turn, so cheaper tiers shrink absolute dollars but not the 48% ratio; prompt caching (cache reads at USD 0.25 to 1 per million tokens [10]) compresses the input side of both columns and slightly increases the relative weight of the displaced output tokens. Second-order savings compound in the same direction: fewer turns mean smaller context windows (the monitoring conversation never grows), fewer opportunities for derailment, and less human re-review of agent-invented procedures. Environment cost is unchanged by the pattern itself (the shell runs on the developer workstation), so token economics dominate the marginal cost of a ticket.

6. Speed and Reliability Analysis

6.1 Latency: Seconds per Turn Versus Milliseconds per Script

An agentic turn costs seconds (model latency plus tool execution), while a shell script costs milliseconds to seconds total. Applying the turn counts of Table 2 with a conservative 20 s per turn, orchestration overhead falls from about 34 min (103 turns) to about 10 min (24 turns + about 2 min of script runtime) per ticket: the ticket spends its wall-clock on implementation and test execution instead of on conversational plumbing. Two shell mechanisms matter disproportionately: detached execution (dispatch returns in about 1 min and the mission runs 30 to 90 min with zero conversational footprint) and the long-poll status reader (blocks up to 8 min server-side, re-checking every 15 s), which detects

completion within 15 s using about 7 calls per hour, a responsiveness that a fixed sleep-and-ask loop only achieves with 4 × the calls.

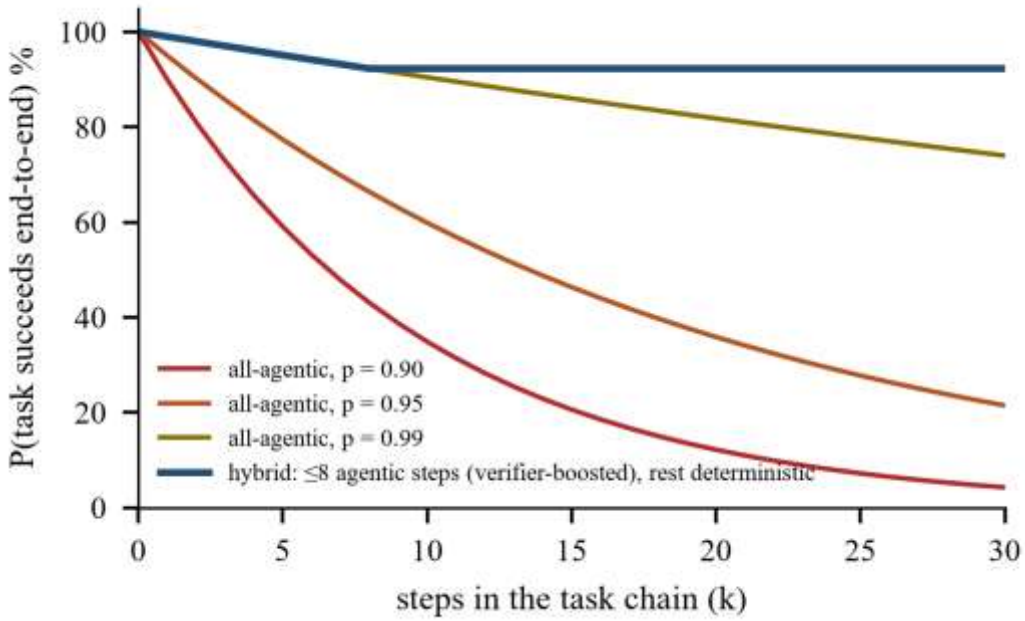
6.2 Reliability: Compounding and the Verifier Boost

Chaining k probabilistic steps of per-step success p gives end-to-end success p^k: at p = 0.95, twenty steps succeed 36% of the time, which is demo territory, not production. The hybrid attacks both factors (Figure 6): deterministic steps take p of about 1 and drop out of the exponent, and the remaining agentic steps sit inside verifier loops whose retries raise effective reliability as given by Equation (2), where r is the retry budget; e.g., p = 0.85 with two verified retries yields p_{eff} of about 0.997.

$$p_{eff} = 1 - (1 - p)^{r+1} \quad (2)$$

This is why delegating execution to interpreters outperforms free-form reasoning [5], why agent-computer-interface design lifts agent success [6], and why the pipeline's missions end in a deterministic verdict (CI green) rather than model self-assessment. Reliability here is also security: guard behaviour must not be probabilistic when the threat is a prompt-injected agent [14, 15]; a git hook blocks a forbidden push with p = 1 regardless of what the model was persuaded to attempt.

Figure 6: End-to-End Success Versus Chain Length



6.3 Reproducibility and Measurability

Determinism also buys the ability to know what happened: every dispatch writes the same briefs, every state transition is a logged file rewrite, every verdict is re-runnable. Given the documented gap between perceived and actual AI productivity [12], an architecture whose overhead, cost, and outcomes are directly measurable is a prerequisite for honest evaluation; the numbers in Section 5 exist because the shell makes them observable.

7. Benefits, Anti-Patterns, and Limitations

Table 3 summarises the benefits quantified or argued above. The pattern also has failure modes. Over-scripting: forcing judgment-laden steps into brittle scripts recreates classic robotic-process-automation fragility; the matrix of Figure 4 exists to prevent both directions of misplacement. Shell maintenance: the

deterministic layer is software; it needs tests and versioning like any other (the production shell treats its scripts as first-class code). Verifier blind spots: CI green proves what the tests assert and nothing more; human review remains the gate for semantic correctness. Scope of evidence: cost figures derive from a parameterised model anchored in one production pipeline's envelope and current list prices [10]; a multi-team controlled measurement is future work.

Table 3: Benefits of the Hybrid Pattern

Dimension	Mechanism	Magnitude (This Analysis)
token cost	orchestration displaced to $0 scripts	about 48% per ticket (Table 2)
wall time	ms scripts + detached missions + long-poll	orchestration overhead −70% (Section 6.1)
reliability	p of about 1 steps + verifier retries (Equation (2))	flattened p^k decay (Figure 6)
security	guards enforced with p = 1, not prompted	policy holds under prompt injection [14]
auditability	logged state transitions, re-runnable verdicts	measurable costs and outcomes (Section 6.3)
human effort	supervision by exception (about 7 calls/h relay)	attention freed for parallel tickets (Section 6.1)

8. Conclusion and Future Work

Agentic AI and deterministic tooling are strongest exactly where the other is weakest: agents supply judgment at token prices and probabilistic reliability; tools supply free, instant, exact execution with no judgment at all. The hybrid architecture analysed here (an agentic core confined to judgment, inside a deterministic shell that dispatches, tracks, guards, verifies, and cleans up) converts that complementarity into measured outcomes: roughly half the token cost per ticket, a two-thirds reduction in orchestration time, exponential-decay reliability replaced by verifier-bounded reliability, and security invariants that hold with probability one. The pattern is consistent with, and gives quantitative teeth to, the emerging industry consensus on building agentic systems [1, 2]. Future work: controlled multi-team cost/outcome measurement; adaptive placement (learning when a step has become specifiable enough to script); shells that auto-generate their own verifiers from specifications; and extending the analysis beyond software engineering to other domains with strong deterministic oracles.

Acknowledgement

The author thanks the engineering colleagues who operate and review the production pipeline referenced in Sections 4 to 6.

References

1. Anthropic, “Building Effective Agents”, December 2024, https://www.anthropic.com/engineering/building-effective-agents
2. OpenAI, “A Practical Guide to Building Agents”, 2025, https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf

3. S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models", International Conference on Learning Representations (ICLR), 2023
4. T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools", Advances in Neural Information Processing Systems 36 (NeurIPS), 2023
5. L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, G. Neubig, "PAL: Program-Aided Language Models", Proceedings of the 40th International Conference on Machine Learning (ICML), PMLR 202, 2023, 10764–10799
6. J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, O. Press, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering", Advances in Neural Information Processing Systems 37 (NeurIPS), 2024
7. C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?", International Conference on Learning Representations (ICLR), 2024
8. OpenAI, "Introducing Codex", May 2025, https://openai.com/index/introducing-codex/
9. GitHub Docs, "About GitHub Copilot Cloud Agent", 2025, https://docs.github.com/en/copilot/concepts/agents/coding-agent/about-coding-agent
10. Anthropic, "Claude Pricing", 2026, https://docs.claude.com/en/docs/about-claude/pricing
11. Stack Overflow, "2025 Developer Survey: AI", 2025, https://survey.stackoverflow.co/2025/ai
12. J. Becker, N. Rush, E. Barnes, D. Rein, "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity", METR, July 2025, https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/
13. Cognition, "Introducing Devin, the First AI Software Engineer", March 2024, https://cognition.ai/blog/introducing-devin
14. OWASP GenAI Security Project, "LLM01:2025 Prompt Injection", 2025, https://genai.owasp.org/llmrisk/llm01-prompt-injection/
15. K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection", Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec), 2023, 79–90
16. BMad Code, "BMAD-METHOD: Breakthrough Method of Agile AI-Driven Development", 2025, https://github.com/bmad-code-org/BMAD-METHOD
17. LangChain, "LangGraph: Balance Agent Control with Agency", 2025, https://langchain-ai.github.io/langgraph/